

FILE PATH TRAVERSAL, TRAVERSAL SEQUENCES STRIPPED WITH SUPERFLUOUS URL-DECODE.

Programación Wap

Prof: Servando Lopez Contreras

David Campos Morado – 183511
Ingeniería en Tecnologías de la Información

Ficha Técnica del Laboratorio

- **Nombre del laboratorio:** File path traversal, traversal sequences stripped with superfluous URL-decode.
- **Nivel:** Practitioner (Intermedio).
- **Tipo de explotación:** File Path Traversal (Evasión de Filtro con Doble Codificación URL).
- **Objetivo del laboratorio:** Aprovechar una vulnerabilidad de salto de directorio burlando un filtro de seguridad mediante la decodificación URL, para lograr leer el archivo `/etc/passwd` y conseguir el estatus de "Lab Solved".

Marco Teórico y Fundamentación Técnica

Como vimos en laboratorios anteriores, el **File Path Traversal** nos permite salirnos de la carpeta pública de la página web para acceder a los archivos internos del servidor. En el OWASP Top 10, esto sigue siendo un problema crítico clasificado como *Broken Access Control*.

Para este laboratorio, los desarrolladores intentaron mejorar su seguridad bloqueando cualquier texto que contenga secuencias de salto como `"../"`. Sin embargo, cometieron un error lógico gravísimo en el **orden de las operaciones**: primero pasan la entrada del usuario por el filtro de seguridad y, *después*, el servidor decodifica el texto (URL-decode).

Este error estructural vuelve a comprometer la **Triada CIA**:

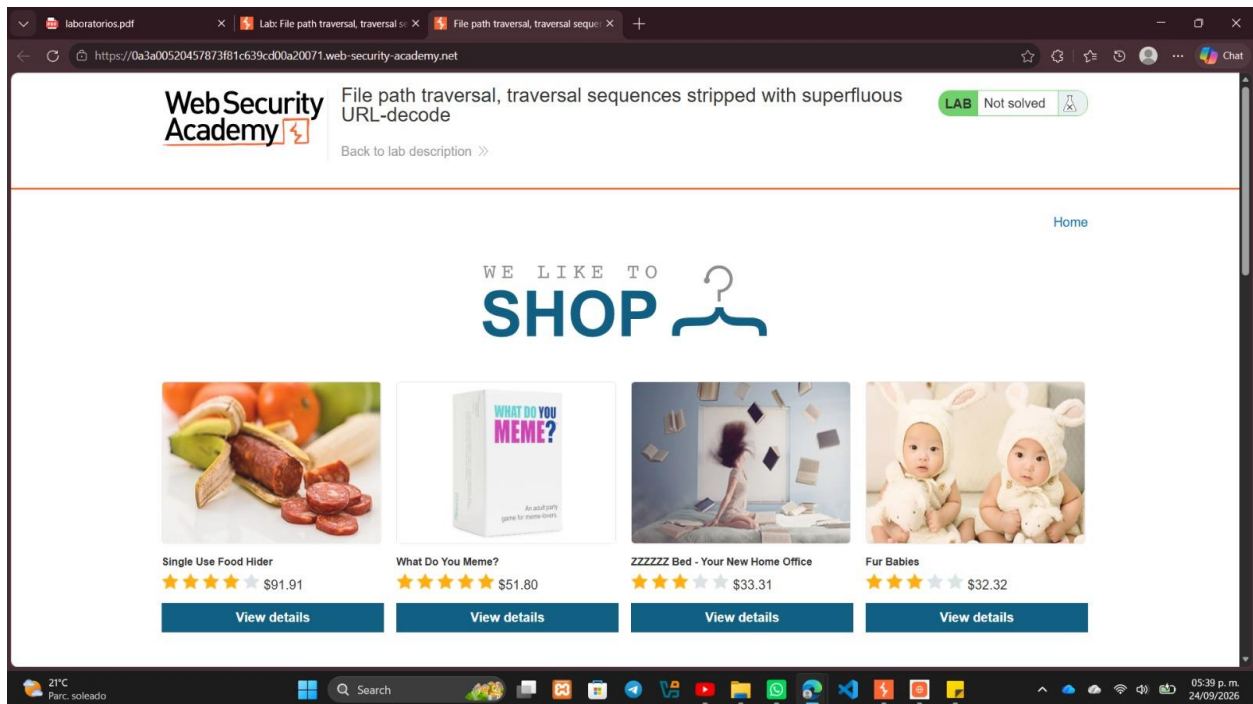
- **Confidencialidad (Crítico):** Al evadir el filtro, tenemos acceso directo de lectura a archivos sensibles del sistema operativo.
- **Integridad (Bajo):** El ataque sigue siendo de solo lectura, por lo que no modificamos la información del servidor.
- **Disponibilidad (Medio):** Podría haber una sobrecarga si se automatizan múltiples peticiones buscando archivos pesados.

Desarrollo Técnico: Procedimiento, Payload y Evidencias

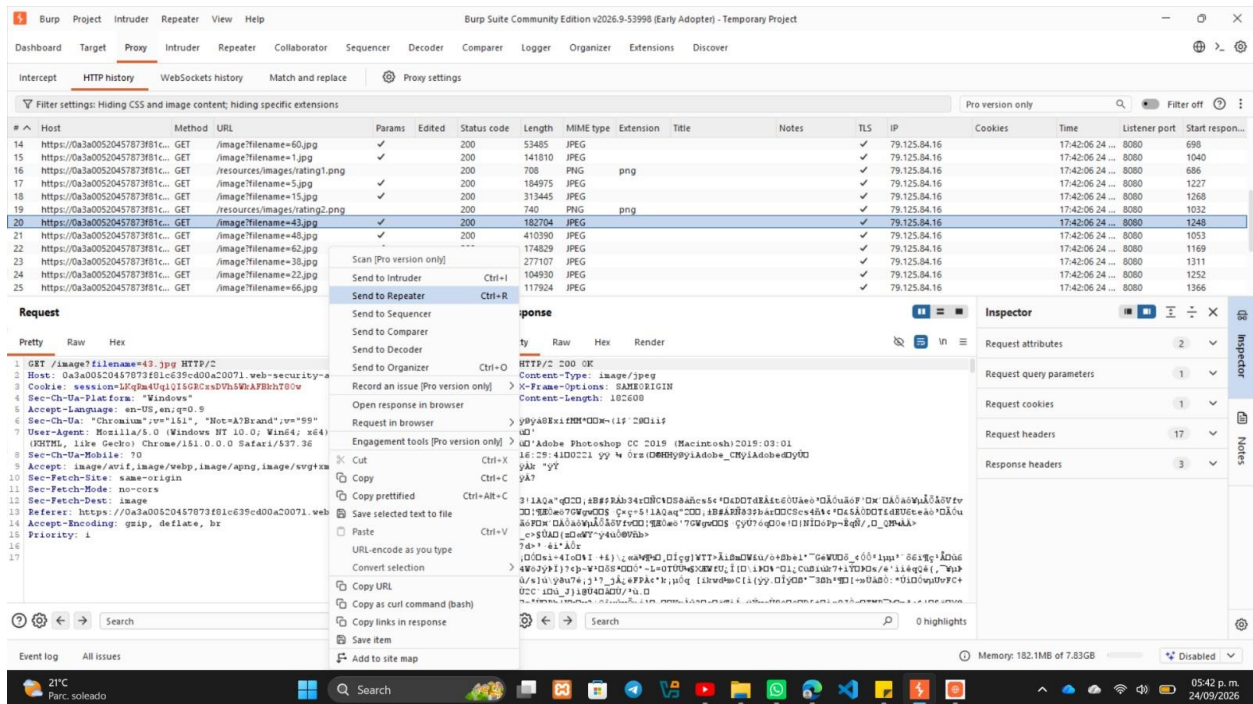
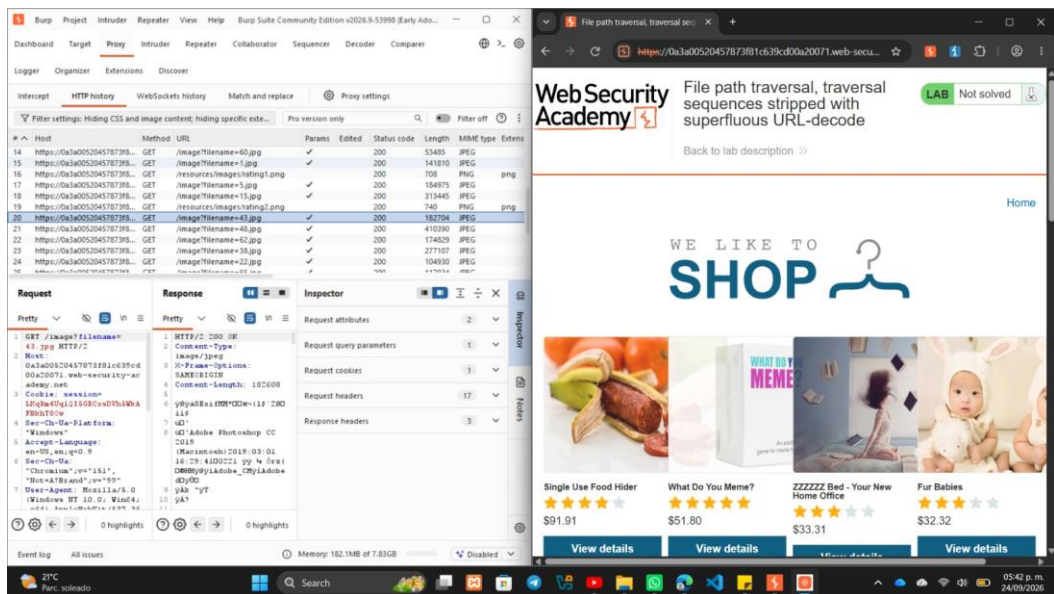
Para resolver este laboratorio, utilicé Burp Suite siguiendo estos pasos y documentando cada intento:

Interceptación del tráfico

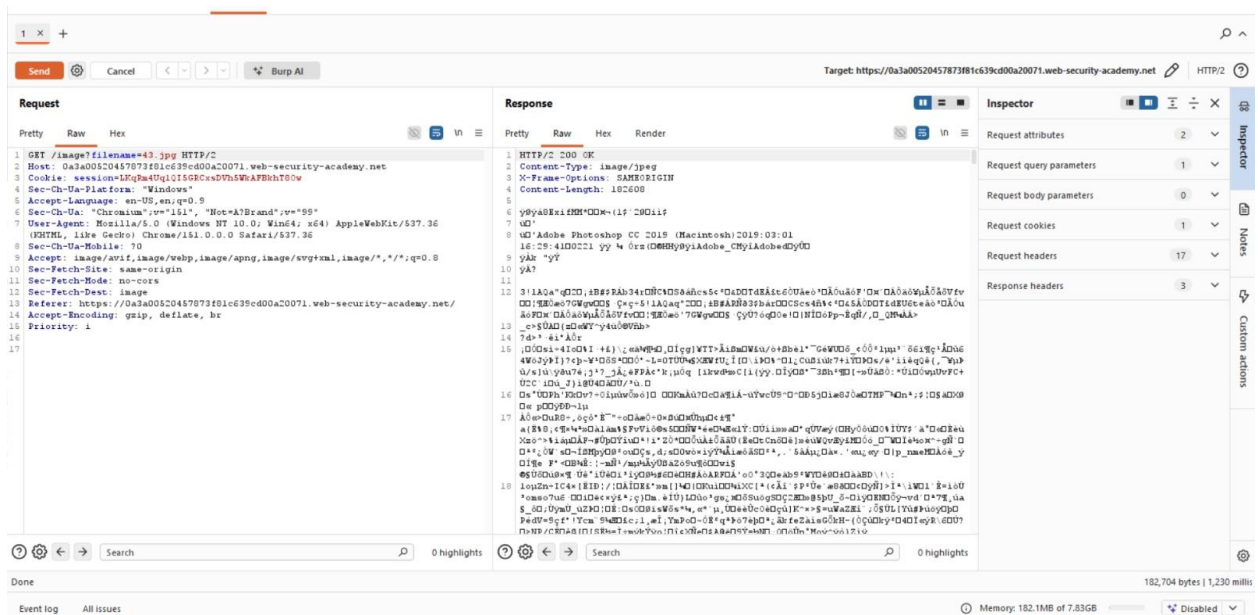
Comencé entrando a la página principal de la tienda, viendo que el laboratorio aún no estaba resuelto.



Al navegar por los productos, capturé el tráfico en el historial HTTP de Burp Suite



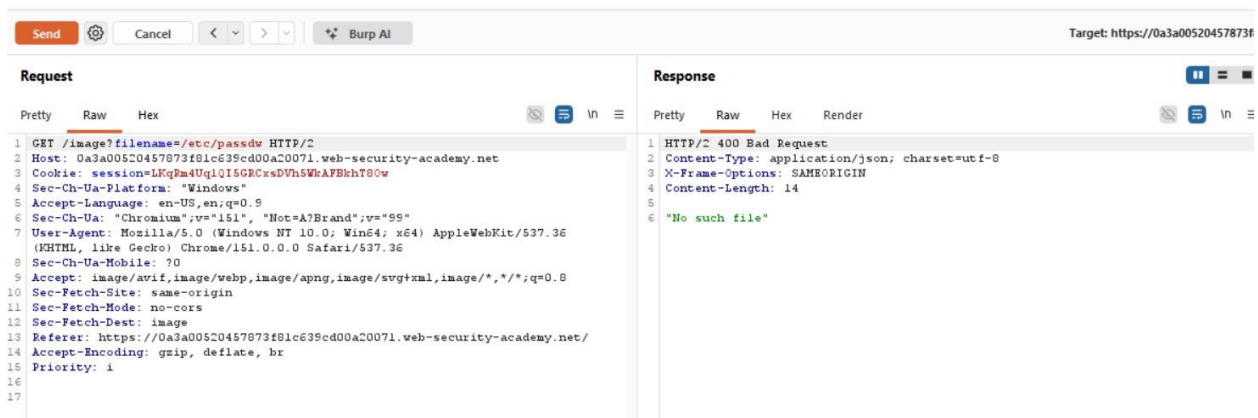
Identifiqué la petición GET que carga las imágenes usando el parámetro filename=43.jpg y la mandé al *Repeater* para verificar que el servidor respondiera con un 200 OK y la imagen normal.



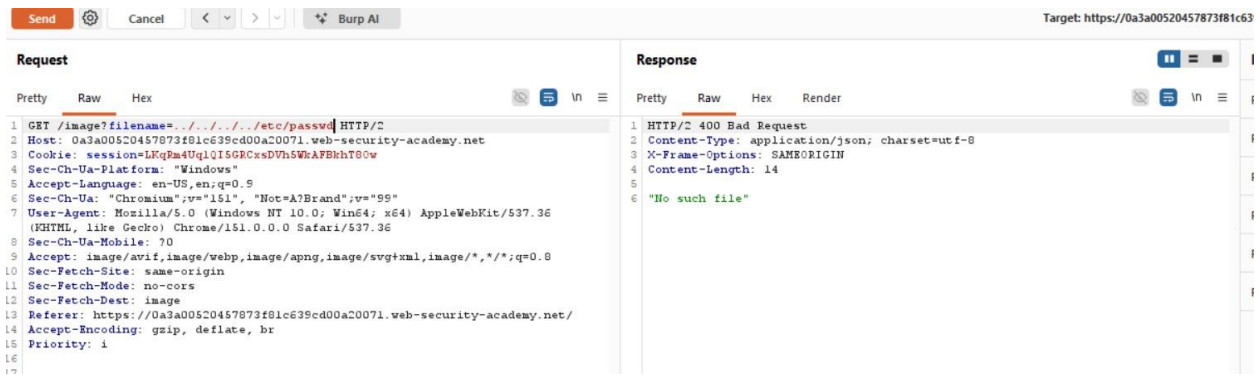
Pruebas de seguridad y bloqueos

Para entender cómo se estaba defendiendo el servidor, hice varias pruebas directas:

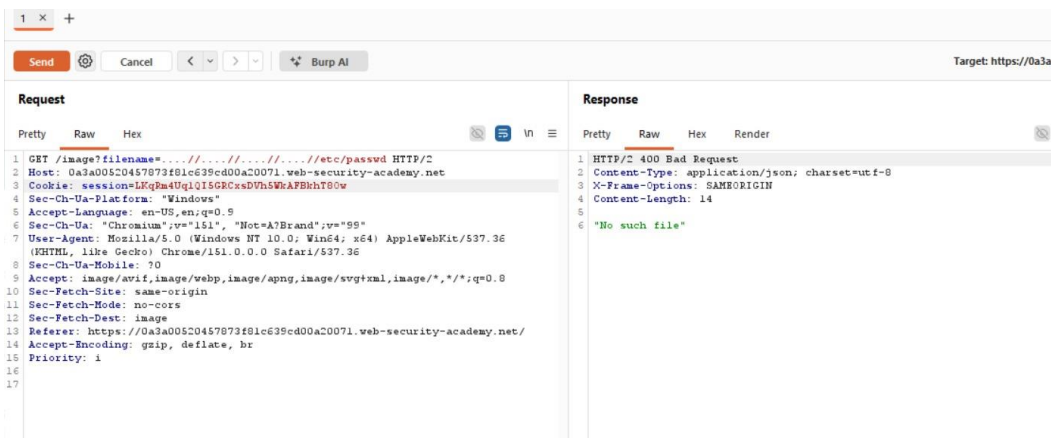
- Intenté leer la ruta absoluta directamente (`filename=/etc/passwd`), pero me devolvió un error 400 Bad Request indicando "No such file".



- Intenté el salto de directorio normal (`filename=.././../etc/passwd`), y también fue bloqueado.

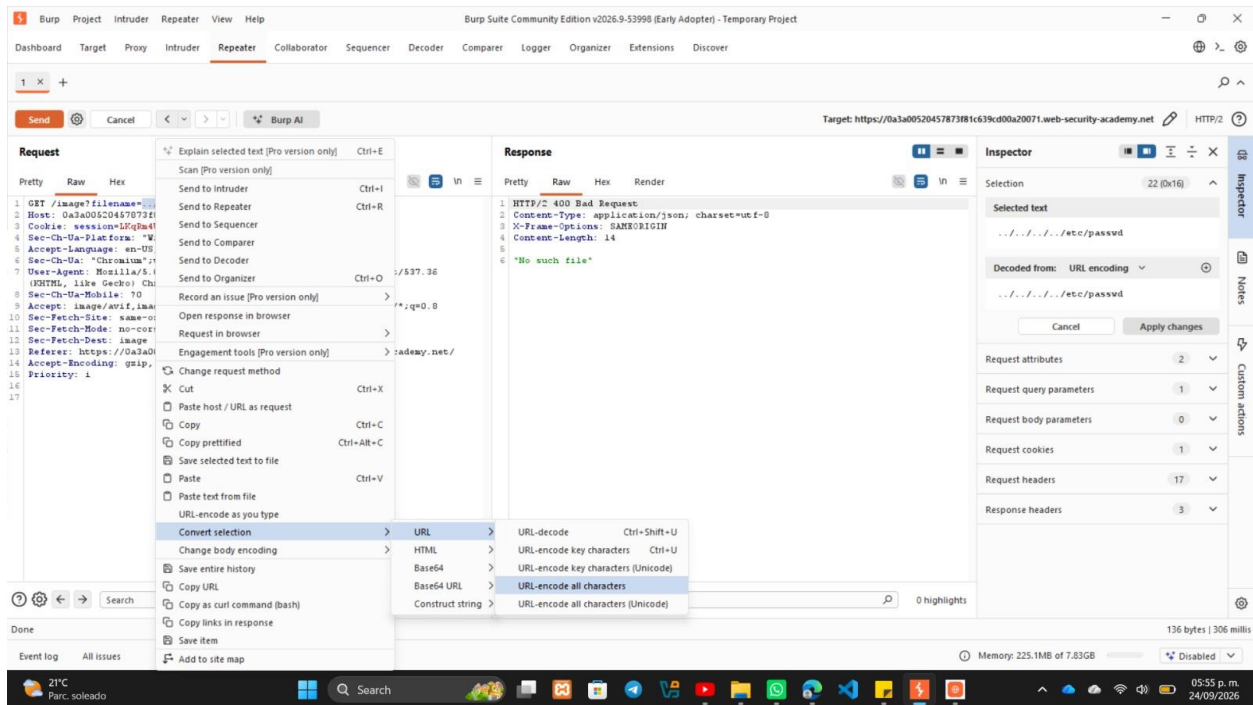


- Incluso probé usar el payload ofuscado del laboratorio anterior (../../../../etc/passwd), pero esta vez el servidor sí estaba preparado y me lo.

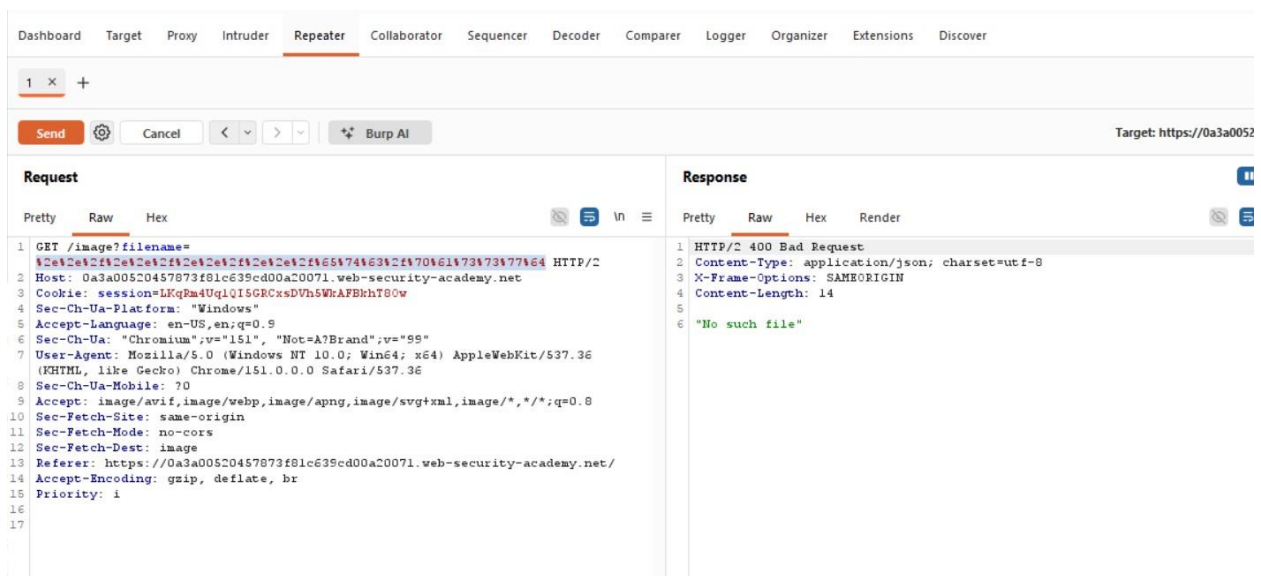


Armado del payload con Doble URL Encoding

Como sabía que la página decodificaba el texto (URL-decode) *después* de filtrarlo, utilicé la herramienta de codificación de Burp Suite.



Primero apliqué una **codificación URL simple** (%2e%2e%2f...), pero el servidor la detectó y me regresó un error 400.



Así que apliqué la técnica de **Doble URL Encoding**. Al codificar por segunda vez, los símbolos de porcentaje (%) de la primera codificación se convierten en %25.

Payload final utilizado:

%252e%252e%252f%252e%252e%252f%252e%252e%252fetc/passwd

La lógica para engañar al sistema es esta:

- Yo envío el texto doblemente codificado.
- El filtro de seguridad revisa la cadena buscando ../ o su versión normal codificada. Como solo ve %25..., dice "aquí no hay ataque" y lo deja pasar libremente.
- Inmediatamente después de pasar la seguridad, el backend de la página hace la decodificación URL y convierte los %25 de regreso a %.
- El servidor web final toma ese %2e%2e%2f resultante, lo interpreta como ../ y ejecuta el salto.

Exfiltración y confirmación

Injecté este payload con doble codificación en el *Repeater* y envié la petición. Gracias al error en el orden de validación, el servidor procesó el salto de directorio de forma invisible y me contestó con un HTTP/2 200 OK, exponiendo en pantalla todo el archivo de usuarios del sistema.

The screenshot displays the Burp Suite interface during a security test. The 'Request' tab on the left shows a GET request to a target URL with a double-encoded payload. The 'Response' tab in the center shows an HTTP/2 200 OK response with a Content-Type of image/jpeg. The 'Inspector' tab on the right shows the response body, which is a list of system users. A context menu is open over the response body, showing options like 'Convert selection', 'Change body encoding', 'Save entire history', 'Copy URL', 'Copy as curl command (bash)', 'Copy links in response', 'Save item', and 'Add to site map'.

actualizando oficialmente el estado de la práctica a "Lab Solved".

Web Security Academy > Path traversal > Lab

Lab: File path traversal, traversal sequences stripped with superfluous URL-decode

PRACTITIONER

 **LAB**  **Solved**

This lab contains a path traversal vulnerability in the display of product images.

The application blocks input containing path traversal sequences. It then performs a URL-decode of the input before using it.

To solve the lab, retrieve the contents of the `/etc/passwd` file.

 **ACCESS THE LAB**

Estrategias de Mitigación Recomendadas

Para solucionar esta vulnerabilidad, los desarrolladores deben corregir el diseño de su validación aplicando estas prácticas:

Orden correcto de operaciones

Nunca se debe validar la entrada del usuario *antes* de decodificarla. El servidor primero debe hacer todas las decodificaciones necesarias (URL-decode) y, una vez que tiene el texto final y real en texto plano, entonces sí pasarlo por el filtro de seguridad.

Canonicalización de rutas

Antes de intentar acceder al archivo físico, el código debe traducir la ruta a su forma absoluta (usando funciones como `realpath`) y verificar estrictamente que la ruta resultante no se salga de la carpeta de imágenes permitida.

No confiar en listas negras

Intentar bloquear cadenas específicas (`../` o sus variantes codificadas) siempre termina en errores y bypasses. Es mucho más seguro usar listas blancas que solo acepten letras, números y una extensión válida (`.jpg`, `.png`).

Conclusión y Aprendizaje Técnico

Este laboratorio me enseñó una lección fundamental: en ciberseguridad, **el orden de las operaciones en el código importa muchísimo**. De nada sirve tener un filtro que detecte ataques si aplicas ese filtro sobre datos ofuscados y después los traduces para usarlos en el sistema.

Burlar este filtro fue cuestión de entender cómo procesa la información el backend. Usar la doble codificación URL (%25...) me demostró que los atacantes siempre buscarán aprovecharse de cómo el sistema interpreta los caracteres especiales.

Me quedo con el aprendizaje de que las validaciones de seguridad siempre deben hacerse como el paso absolutamente final antes de tocar el sistema de archivos, asegurándonos de que la cadena de texto ya no va a sufrir ninguna otra transformación.